

impactgraph

AI-powered Change Impact Graph for data and code systems

Complete project documentation — from idea to open-source-ready library

Version 0.1.0

Location: C:\Users\Hp\Documents\Innovation\impactgraph

Built with Claude Code (Claude Fable 5) · August 2026

License: MIT · Python ≥ 3.9 · pip install impactgraph

Contents

1. Executive summary	4
2. The brief and the design principle	4
3. How it was built — the A-to-Z timeline	5
4. Skills, tools and technologies used	6
4.1 Claude Code skills and tools	6
4.2 Python libraries and standards	7
5. Architecture	8
5.1 Pipeline	8
5.2 Package layout	8
5.3 Data model	9
5.4 Node id conventions	9
5.5 Risk scoring and test recommendations	9
5.6 The AI layer	10
6. Worked example — the demo	11
7. How to use it	11
7.1 Install	11
7.2 Command line	11
7.3 Python API	12
8. Testing — what was verified and how	13
8.1 Automated suite (34 tests, all passing)	13
8.2 End-to-end verification from the built wheel	13
8.3 Bugs found and fixed along the way	14
8.4 Honest remaining gaps	14
9. How impactgraph differs from what already exists	15
9.1 What is genuinely different	15
9.2 Where existing tools are ahead (known gaps)	15
10. Packaging and open-source readiness	16
10.1 Packaging	16
10.2 Repository hygiene	16
10.3 Publishing to GitHub (next step)	16
11. Roadmap	16
Appendix A — Command reference	18
Appendix B — File inventory	18
Appendix C — Design decisions worth remembering	18

(In Microsoft Word: right-click the table of contents and choose Update Field if page numbers are missing.)

1. Executive summary

impactgraph is a pip-installable Python library and command-line tool that answers the question every data-platform and backend engineer asks before merging a change:

The question

“If I change this file, function, dbt model, SQL column, API, or table — what can break?”

It builds one unified dependency graph that spans both the code world (Python files, functions, imports, calls) and the data world (dbt models, sources, warehouse tables, columns, dashboards), and it does so deterministically — from real engineering artifacts, never from an LLM’s guess. Graph algorithms then compute the blast radius of a change, a risk level, and a concrete test plan. An optional AI layer explains the computed result in plain language for the engineer about to merge.

Headline facts

- **Name:** impactgraph — verified free on PyPI before building (impactlens was also free).
- **Scope delivered:** 4 deterministic extractors (Python AST, dbt manifest.json, SQL via sqlglot, git diff), unified graph with direction-aware impact propagation, risk scoring, test recommender, rich terminal report, CLI with 5 commands, optional Claude-powered explanation layer.
- **Quality:** 34 automated tests passing; end-to-end verified from the built wheel in a clean virtual environment; one real Windows bug (UTF-8 BOM) found and fixed during that pass.
- **Open-source readiness:** MIT license, README, CONTRIBUTING guide, .gitignore, GitHub Actions CI (Ubuntu + Windows × Python 3.9/3.11/3.13), wheel + sdist built, two commits on a local main branch, ready to push.

2. The brief and the design principle

The project started from a product idea: existing tools already cover pieces of the problem — Graphify and Code-Graph-RAG build graphs from source code, while DataHub and OpenLineage focus heavily on data lineage — but nothing connects both worlds into a single “what breaks if I change this” answer. The target experience was a chain such as:

Git change → Python function → Lambda → API → S3 → Snowflake table → dbt model → report / dashboard

and a report such as:

⚠ Change Impact

Changed: models/customer.sql (customer_id → customer_key)

Potential impact:

customer.sql → dim_customer → fact_booking → { revenue_report, customer_dashboard }

Risk: HIGH

Affected: 3 dbt models, 2 Snowflake tables, 1 Python function, 2 reports

Recommended tests:

✓ dbt relationship test ✓ fact_booking integration test ✓ customer API contract test

The one non-negotiable design principle

The LLM never creates the fundamental graph. The graph is built deterministically from actual engineering artifacts — Python AST, SQLGlot, dbt manifest.json, git diff, (later) Snowflake metadata and Airflow/Lambda definitions — then graph algorithms compute impact, and only then does AI produce the explanation, risk narrative, and test plan.

This gives far higher trust than asking a model to guess relationships, and it is the line that separates impactgraph from LLM-first code-graph tools.

3. How it was built — the A-to-Z timeline

The library was built in a single autonomous Claude Code session. Below is the actual sequence of work, including the detours, so the process is reproducible.

Step	What happened	Why it mattered
1. Load tooling	Loaded the claude-api skill (authoritative guidance for the Anthropic SDK) and the WebFetch tool.	The AI layer calls the Claude API; the skill supplies current model IDs, streaming and refusal-handling rules instead of relying on memory.
2. Name check	Queried pypi.org/pypi/<name>/json for impactlens and impactgraph — both returned 404 (available).	A pip-installable library needs a free name; impactgraph matched the pitch literally.
3. Architecture	Designed a src-layout package: graph/ (model + graph), extractors/, analysis/, ai/, report.py, cli.py.	Clear separation: deterministic extraction → graph algorithms → AI explanation.
4. Data model	Defined NodeType / EdgeType enums, Node and Edge dataclasses, and the IMPACT_DIRECTION table that says which way change flows for each edge type.	This table is the heart of the library — it makes impact propagation correct without special-casing.
5. Graph engine	ImpactGraph over networkx MultiDiGraph: add/merge, resolve references, BFS impact() with depth, impact_tree(), impact_paths(), JSON save/load.	One engine serves every extractor and the CLI.
6. Extractors	Python AST extractor; dbt manifest extractor; sqlglot SQL extractor (optional extra); git diff extractor that maps changed line ranges to functions.	Each emits a graph fragment using shared node-id conventions so fragments merge.
7. Analysis	Deterministic risk scoring (type weights + depth discount + breadth bonus) and a rule-	Produces the opinionated output lineage tools lack.

Step	What happened	Why it mattered
	based test recommender; analyze_impact() bundles blast radius + risk + tests + trees.	
8. AI layer	explain_impact() using the Anthropic SDK: streaming, claude-opus-5, refusal handling, helpful ImportError when the [ai] extra is missing.	AI explains, never constructs.
9. Report + CLI	rich-based tree/panel renderer; argparse CLI with build / impact / diff / nodes / explain.	Usable from a terminal and from CI.
10. Tests	28 pytest tests with synthetic fixtures (tiny Python project, minimal dbt manifest, SQL files, a throwaway git repo).	Offline, fast (≈ 3 s), covers every extractor and the CLI.
11. Demo	examples/demo: dbt manifest + Python API + one bridging edge; ran impactgraph impact models/customer.sql.	Reproduced the pitch output end-to-end; surfaced a resolve() ambiguity bug (fixed).
12. Packaging	python -m build produced wheel + sdist; editable install with [dev] extra.	Confirms pip install works.
13. Open-source prep	.gitignore, GitHub Actions CI, CONTRIBUTING.md, git init + initial commit.	Ready for a public repo.
14. Deep test pass	Fresh venv, install the built wheel, run build/diff/explain on a real git repo written by PowerShell.	Found the UTF-8 BOM bug (extractor silently skipped every file) — fixed in all three extractors with regression tests; hardened CLI explain; added offline AI-layer tests. Test count $\rightarrow$ 34. Second commit.

4. Skills, tools and technologies used

4.1 Claude Code skills and tools

Skill / tool	Used for
claude-api skill	Authoritative Anthropic SDK guidance: current model ID (claude-opus-5), default to streaming with get_final_message(), adaptive thinking defaults, handling stop_reason == "refusal", zero-argument Anthropic() client credential resolution.
WebFetch	PyPI JSON API lookups to confirm the package name was free.
Write / Edit	Creating the ~ 30 source, test, and documentation files; surgical fixes (resolve() priority, BOM reads, CLI error handling).
PowerShell / Bash	pip install -e, pytest, python -m build, venv creation, git init/commit, the end-to-end CLI runs, Word automation for the PDF export of this document.

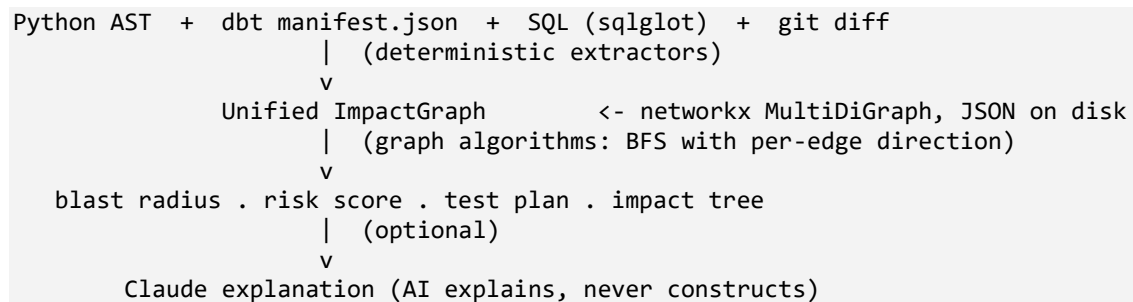
Skill / tool	Used for
docx skill	This document (docx-js generation, then conversion to PDF).
Memory	Saved a project memory note so future sessions know where the library lives and its design rules.

4.2 Python libraries and standards

Technology	Role in impactgraph	Core / optional
ast (stdlib)	Parse Python files: functions, classes, imports, call sites, line spans.	core
networkx ≥ 3.0	MultiDiGraph storage, traversal, all_simple_paths for impact_paths().	core
rich ≥ 13	Terminal panel + tree rendering of the impact report.	core
argparse (stdlib)	CLI with five sub-commands.	core
subprocess + git	git diff --name-only and --unified=0 hunk parsing.	core (git on PATH)
sqlglot ≥ 20	Parse CREATE TABLE/VIEW ... AS SELECT and INSERT statements into table lineage and output columns.	[sql] extra
anthropic ≥ 0.40	Claude API client for the explanation layer.	[ai] extra
pytest ≥ 7	Test suite.	[dev] extra
setuptools + build	PEP 621 pyproject.toml, src layout, console_scripts entry point, wheel + sdist.	build
GitHub Actions	CI matrix (Ubuntu + Windows × Python 3.9/3.11/3.13) plus a build job that uploads dist/.	CI

5. Architecture

5.1 Pipeline



5.2 Package layout

Path	Responsibility
src/impactgraph/__init__.py	Public API re-exports and __version__.
graph/model.py	NodeType, EdgeType, IMPACT_DIRECTION, Node, Edge dataclasses (+ to/from dict).
graph/graph.py	ImpactGraph: add/merge, find/resolve, impact(), impact_tree(), impact_paths(), save/load.
extractors/base.py	Extractor abstract base class — extract() -> ImpactGraph.
extractors/python_extractor.py	Files, functions, classes, CONTAINS, IMPORTS (resolved to project files), best-effort CALLS.
extractors/dbt_extractor.py	Models/seeds/snapshots, sources, exposures, DEPENDS_ON DAG, WRITES_TO tables, declared columns, SQL-file CONTAINS.
extractors/sql_extractor.py	sqlglot lineage: table/view nodes, DEPENDS_ON sources, output COLUMN nodes.
extractors/git_extractor.py	collect_changes() (files + line ranges) and changed_node_ids() (map to file/function/model nodes).
analysis/impact.py	analyze_impact() and the ImpactAnalysis dataclass.
analysis/risk.py	Deterministic risk scoring.
analysis/tests_recommender.py	Rule-based test plan.
ai/explain.py	explain_impact() via the Claude API.
report.py	rich rendering.
cli.py	impactgraph build / impact / diff / nodes / explain.
tests/	34 pytest tests across 8 modules.
examples/demo/	Manifest + Python API + build_demo.py reproducing the pitch.

Path	Responsibility
.github/workflows/ci.yml	CI matrix and build artifact.

5.3 Data model

Node types: file, module, function, class, dbt_model, dbt_source, dbt_seed, dbt_snapshot, exposure, table, view, column, lambda, api, dag, report, dashboard.

Edge types are semantic, and each declares the direction in which change propagates. This is what lets a single BFS give correct answers across code and data:

Edge type	Semantic direction (src → dst)	Impact flows	Meaning
contains	file → function / model → column	forward	changing a file affects everything defined in it
calls	caller → callee	reverse	changing the callee affects its callers
imports	importer → imported	reverse	changing a module affects importers
depends_on	downstream → upstream	reverse	changing upstream data affects dependents
writes_to	model → table	forward	changing a model affects its materialized table
exposes	model → dashboard	forward	changing a model affects the dashboard

5.4 Node id conventions

Every extractor uses the same id scheme, so fragments from different sources merge into one graph and a git-changed path lines up with a dbt model defined in that file:

file:models/customer.sql	func:src/api.py::customers_endpoint
class:src/models.py::Customer	dbt:dim_customer
source:raw.customers	exposure:revenue_report
table:prod.analytics.customer	column:dim_customer.customer_id

5.5 Risk scoring and test recommendations

Risk is deterministic. Each affected node contributes a weight by type — dashboards/reports/exposures 8, APIs 6, lambdas 4, tables/views/dbt models/DAGs 3, seeds/sources 2, functions/classes/columns/files 1 — multiplied by a depth discount (direct hits count fully; deeper hits never below 50 %), plus a breadth bonus for many direct hits (capped at 10). Thresholds: LOW < 6 ≤ MEDIUM < 18 ≤ HIGH < 40 ≤ CRITICAL.

The test recommender maps affected node types to actions: `dbt build --select <model>+` for models, `relationship/not_null` tests for columns, `schema/contract` checks for tables, unit tests for functions, API contract tests, lambda integration tests, manual dashboard validation, DAG test runs.

5.6 The AI layer

`explain_impact(analysis)` serialises the computed `ImpactAnalysis` to JSON and streams a request to Claude (default model `claude-opus-5`, up to 16 000 output tokens) with a system prompt that forbids inventing nodes or dependencies and asks for: what changed, what can break and why (walking the paths), whether the risk level is right, and what to verify before and after deploy. It handles `stop_reason == "refusal"` and raises a clear `ImportError` pointing to `pip install impactgraph[ai]` when the SDK is absent. Credentials resolve from `ANTHROPIC_API_KEY` or an ant auth login profile.

6. Worked example — the demo

examples/demo contains a dbt manifest (raw sources → customer → dim_customer → fact_booking → two dashboards) and a small Python service (booking_api.py with fetch_bookings and bookings_endpoint). build_demo.py merges both extractors and adds a single bridging edge:

```
graph.add_edge(Edge(
    src="func:booking_api.py::fetch_bookings",
    dst="table:prod.analytics.fact_booking",
    type=EdgeType.DEPENDS_ON,
))
```

Running impactgraph impact models/customer.sql then produced:

```
Change Impact          Changed: models/customer.sql          Risk: HIGH (score 35.0)

models/customer.sql (file)
`-- customer (dbt_model) via contains
    |-- prod.analytics.customer (view) via writes_to
    |-- customer_id / email / created_at (columns) via contains
    `-- dim_customer (dbt_model) via depends_on
        |-- prod.analytics.dim_customer (table) via writes_to
        `-- fact_booking (dbt_model) via depends_on
            |-- prod.analytics.fact_booking (table) via writes_to
            |   `-- fetch_bookings (function) via depends_on
            |       `-- bookings_endpoint (function) via calls
            |-- revenue_report (dashboard) via exposes
            `-- customer_dashboard (dashboard) via exposes

Affected: 8 columns . 3 dbt models . 2 tables . 2 dashboards . 2 functions . 1
view
Recommended tests: dbt build --select customer+ dim_customer+ fact_booking+ ;
schema/contract checks on the 3 relations; unit tests for booking_api.py;
validate both dashboards
```

This is exactly the pitch: a change to one SQL file propagates through the dbt DAG, into warehouse relations, across the code/data bridge into a Python API, and out to the dashboards.

7. How to use it

7.1 Install

```
pip install impactgraph          # core: Python + dbt + git extractors
pip install impactgraph[sql]     # + SQL lineage via sqlglot
pip install impactgraph[ai]      # + Claude explanations
pip install impactgraph[all]     # everything
# from the local checkout today:
pip install -e "C:\Users\Hp\Documents\Innovation\impactgraph[all]"
```

7.2 Command line

Command	What it does
<code>impactgraph build --repo ./src --dbt-manifest target/manifest.json --sql ./sql -o impactgraph.json</code>	Run the extractors and save the unified graph.
<code>impactgraph impact dbt:customer</code>	Blast radius, risk and test plan for one or more nodes (ids, names or paths). --json for machine output, --max-depth to limit.
<code>impactgraph diff --repo . --graph impactgraph.json</code>	Blast radius of the current uncommitted change (or --base/--head refs) — maps changed line ranges to the exact functions touched. The CI-friendly command.
<code>impactgraph nodes --search customer --type dbt_model</code>	List / search graph nodes.
<code>impactgraph explain dbt:customer</code>	Report plus a Claude-written explanation (needs [ai] + credentials).

7.3 Python API

```

from impactgraph import ImpactGraph, PythonExtractor, DbtExtractor,
analyze_impact

graph = ImpactGraph()
graph.merge(PythonExtractor("./src").extract())
graph.merge(DbtExtractor("target/manifest.json").extract())

analysis = analyze_impact(graph, ["dbt:customer"])
print(analysis.risk)           # {'score': 24.5, 'level': 'HIGH'}
print(analysis.affected)      # {node_id: depth, ...}
print(analysis.recommended_tests)

from impactgraph.ai import explain_impact # optional
print(explain_impact(analysis))

```

8. Testing — what was verified and how

Testing happened in two passes. The first pass was the unit suite written alongside the code; the second was a deliberate “did you test it completely?” pass that installed the built wheel into a clean environment and exercised the real CLI on a real git repository. The second pass is where the most valuable bug was found.

8.1 Automated suite (34 tests, all passing)

Module	Tests	What it pins down
test_python_extractor.py	5	files/functions extracted; import and call edges propagate in reverse; file change hits contained functions; unrelated functions untouched.
test_dbt_extractor.py	8	models/sources/exposures; downstream propagation; SQL file → model; source change hits everything; upstream not affected; materialized tables linked; risk + dbt build recommendation; resolve by name.
test_sql_extractor.py	3	table lineage through two CREATE TABLE AS SELECT files; output columns; downstream-only.
test_graph.py	6	merging code + data graphs with a bridge edge; save/load round-trip; impact tree shape; impact paths; max_depth; JSON-serialisable analysis.
test_git_extractor.py	2	uncommitted change maps to the exact function (and not its neighbour); diff impact reaches the caller.
test_cli.py	4	build + impact --json; unknown node → exit 2; missing graph → exit 2; nodes listing.
test_bom_files.py	3	UTF-8 BOM files parse in all three extractors (regression).
test_ai_layer.py	3	helpful ImportError without anthropic; request payload contains the deterministic analysis and the no-invention system prompt; refusal handled — all against a stub SDK, offline.

8.2 End-to-end verification from the built wheel

1. Built wheel + sdist with `python -m build`.
2. Created a fresh venv, installed the wheel with the [sql] extra, confirmed the console_scripts entry point metadata (`impactgraph = impactgraph.cli:main`) and imports.
3. Created a throwaway git repo with PowerShell-written Python files and the demo manifest; ran `impactgraph build`; edited one function; ran `impactgraph diff`.
4. Result after fixes: the diff command detected the uncommitted change, flagged `db.py` and `load_customers` specifically (not `load_orders`), and propagated to `customers_endpoint` via the call edge — Risk MEDIUM, exit 0.

5. Verified impactgraph explain exits 2 with a clear message (no traceback) when the [ai] extra is absent.

8.3 Bugs found and fixed along the way

Bug	How it was found	Fix
resolve() treated a path shared by a file node and the dbt model defined in that file as ambiguous.	Running the demo CLI.	Resolution order: exact id → exact name → exact path → file node on that path → unique substring.
UTF-8 BOM (default for PowerShell / many Windows editors) made ast.parse fail; the Python extractor silently skipped every file (“0 nodes”).	Wheel-based end-to-end run — unit fixtures write BOM-less files so pytest never saw it.	All extractors read utf-8-sig; three regression tests added.
impactgraph explain dumped a raw traceback when anthropic was missing.	Same end-to-end pass.	Catch ImportError, print guidance, exit 2.
Stale wheel: an e2e run used a wheel built before a fix.	Process slip during testing.	Rebuild before every reinstall; noted as a lesson.

8.4 Honest remaining gaps

- **Live Claude API call:** not executed — no ANTHROPIC_API_KEY on the build machine. Everything around the call is pinned by stub tests; one run of impactgraph explain with a key closes it.
- **Python 3.9–3.13:** the build machine only has 3.14; the CI matrix covers the older versions on first push.
- **Console .exe launcher:** this machine’s security policy blocks every pip-generated .exe (environmental, not a packaging defect); verified via entry-point metadata and python -m impactgraph.cli.

9. How impactgraph differs from what already exists

Parts of this problem are well covered by existing tools. The honest positioning is that impactgraph is a complement to the lineage platforms, not a replacement — the fast, local, PR-time layer that also sees your code.

Tool / category	What it does well	Where it stops	impactgraph's difference
DataHub, OpenLineage, Marquez	Enterprise data lineage across warehouses, pipelines, BI; runtime-collected; DataHub has an impact view.	Data plane only — no application code; a platform to deploy and ingest into, not a 5-second PR check.	Adds the code plane and the diff-to-function entry point; runs locally from pip in CI.
dbt (dbt ls --select model+)	Model-level lineage and exposures inside one dbt project.	Ends at the dbt boundary; no Python callers, no git-diff mapping, no risk or test plan.	Merges dbt lineage with code, scores risk, recommends tests.
Graphify, Code-Graph-RAG, Sourcegraph-style	Build graphs from source code, usually to feed an LLM for code Q&A.	Code plane only; the LLM often participates in interpreting the graph.	Deterministic graph; crosses into tables and dashboards.
sqllineage, sqlglot, SQLMesh	SQL parsing and (for SQLMesh/DataHub) real column-level lineage.	A layer, not a PR-time impact tool.	impactgraph uses sqlglot for this layer; adds everything above it.

9.1 What is genuinely different

1. **One graph spanning both worlds.** A git diff on a Python file and a dbt model change land in the same graph, so impact propagates code → table → dbt → dashboard and back.
2. **Diff-to-function granularity as the entry point.** The unit of analysis is the change you are about to merge: the diff's line ranges are mapped to the exact functions touched, not just "this file changed".
3. **Developer-workflow shape.** pip install, a JSON graph, one command in CI. No server, no ingestion pipeline, no metadata service.
4. **Deterministic graph / AI explanation split.** The graph, blast radius, risk and test plan are computed; the model only narrates an analysis it cannot alter — LLM readability with graph-level trust.

9.2 Where existing tools are ahead (known gaps)

- **Coverage:** lineage platforms see runtime lineage across dozens of systems; impactgraph sees what four extractors can parse. Python call resolution is name-based (no type inference), and code↔data bridge edges are currently declared by hand — auto-detecting them is the highest-value roadmap item.

- **Column-level lineage:** output columns only today; no column-to-column propagation through transformations.
- **Freshness:** a stored graph goes stale; mitigated by rebuilding in CI, but lineage platforms update continuously.

10. Packaging and open-source readiness

10.1 Packaging

- PEP 621 pyproject.toml, setuptools backend, src layout, requires-python ≥ 3.9 , MIT license, classifiers and keywords set.
- Core dependencies kept light (networkx, rich); heavy ones behind extras: [sql] sqlglot, [ai] anthropic, [all], [dev] pytest.
- console_scripts entry point: impactgraph = impactgraph.cli:main.
- dist/ contains impactgraph-0.1.0-py3-none-any.whl and impactgraph-0.1.0.tar.gz; publishing is `python -m twine upload dist/*`.

10.2 Repository hygiene

- README.md — pitch, install, CLI and Python quick starts, propagation rules, id conventions, roadmap.
- CONTRIBUTING.md — setup, the three project principles, a recipe for adding an extractor, PR rules.
- LICENSE (MIT), .gitignore, .github/workflows/ci.yml (test matrix + build artifact).
- Git history: a452a29 “Initial release: impactgraph v0.1.0”; ced063a “Fix BOM handling in extractors; harden CLI explain; add AI-layer tests”.

10.3 Publishing to GitHub (next step)

The GitHub CLI is not installed on the build machine, so the repo has not yet been pushed. Either:

1. Install GitHub CLI (`winget install --id GitHub.cli -e`), run `gh auth login`, then `gh repo create impactgraph --public --source . --push`; or
2. Create an empty public repo on github.com, then `git remote add origin https://github.com/<you>/impactgraph.git` and `git push -u origin main` (a browser sign-in appears on first push).

Before announcing: replace the placeholder Homepage URL in pyproject.toml and <owner> in CONTRIBUTING.md with the real GitHub username, and consider adding the demo output as a screenshot near the top of the README.

11. Roadmap

Item	Value	Effort
Auto-detect code $\leftrightarrow$ data bridge edges (SQL strings in Python, warehouse connector / boto3 calls)	Highest — removes the one manual step	Medium

Item	Value	Effort
Column-level lineage through dbt compiled SQL	High for rename / type-change impact	Medium–high
OpenLineage / DataHub import	Inherit enterprise lineage; position as complement	Medium
Airflow DAG and AWS Lambda extractors	Completes the pitched chain	Medium
GitHub Action posting the impact report on every PR	Turns users into daily users	Low
Snowflake metadata extractor (information_schema)	Real tables/columns without a manifest	Medium

Appendix A — Command reference

```
# development
pip install -e ".[dev]"          # install with tests + sqlglot
python -m pytest -q              # 34 tests
python -m build                  # wheel + sdist into dist/

# demo
cd examples/demo && python build_demo.py
impactgraph impact models/customer.sql --graph impactgraph.json

# everyday
impactgraph build --repo ./src --dbt-manifest target/manifest.json -o
impactgraph.json
impactgraph diff --repo . --graph impactgraph.json
impactgraph impact dbt:customer --json
impactgraph nodes --search booking
impactgraph explain dbt:customer --model claude-opus-5
```

Appendix B — File inventory

Source (src/impactgraph): `__init__.py`, `cli.py`, `report.py`, `graph/{__init__,model,graph}.py`, `extractors/{__init__,base,python_extractor,dbt_extractor,sql_extractor,git_extractor}.py`, `analysis/{__init__,impact,risk,tests_recommender}.py`, `ai/{__init__,explain}.py`.

Tests: `conftest.py`, `test_python_extractor.py`, `test_dbt_extractor.py`, `test_sql_extractor.py`, `test_graph.py`, `test_git_extractor.py`, `test_cli.py`, `test_bom_files.py`, `test_ai_layer.py`.

Project: `pyproject.toml`, `README.md`, `CONTRIBUTING.md`, `LICENSE`, `.gitignore`, `.github/workflows/ci.yml`, `examples/demo/{manifest.json, app/booking_api.py, build_demo.py}`, `docs/` (this document).

Appendix C — Design decisions worth remembering

- **Semantic edges + a direction table** instead of storing edges in impact direction: keeps extractors natural (“model depends on source”) and the traversal correct.
- **Placeholder nodes on dangling edges** (a dbt model may depend on a node that a later extractor defines): merging never loses an edge.
- **Name-based call resolution** is deliberately best-effort; false positives are preferable to missed impact for a safety tool, and the tree shows “via calls” so reviewers can judge.
- **No LLM in the graph** — tests assert the exact prompt payload so the AI layer can never quietly become load-bearing.